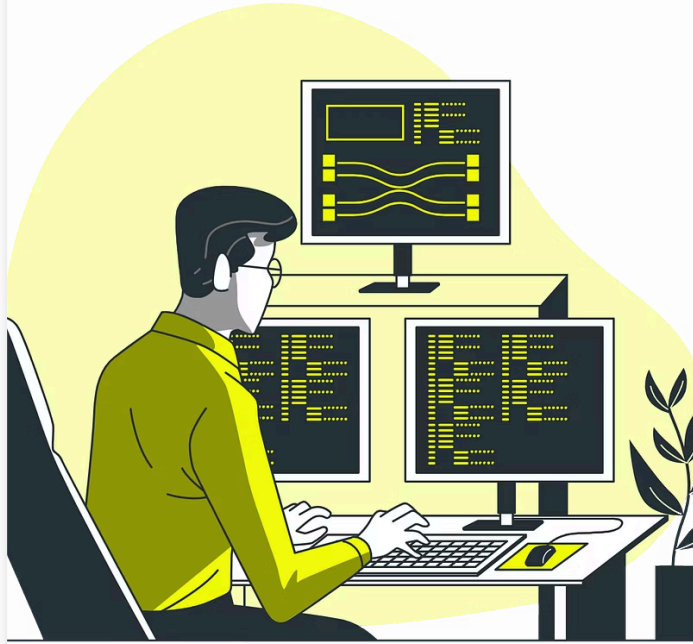




Java

Clase 09 | Introducción a Git y Control de Versiones

¡Les damos la bienvenida! 🚀

🎥 Vamos a comenzar a grabar la clase.

Índice

Clase 09

Clase 10

Clase 9 | Introducción a Git y Control de Versiones

- Conceptos fundamentales de control de versiones.
- Instalación y configuración de Git.
- Flujo de trabajo básico con Git:
 - Creación de repositorios.
 - Comandos básicos: add, commit, push, pull
- Colaboración en proyectos mediante GitHub.
- Buenas prácticas de versionado.

Clase 10 | Introducción a Spring Boot

- ¿Qué es Spring Boot y por qué utilizarlo?
- Creación de un proyecto Spring Boot desde cero.
- Estructura básica de un proyecto Spring Boot.
- Configuración inicial y dependencias necesarias.
- Primeros pasos con Spring Boot.

Objetivos de la Clase



Comprender el control de versiones

Entender su propósito y beneficios en proyectos de software.



Instalar y configurar Git

Aprender a configurar la herramienta principal de control de versiones.



Dominar el flujo de trabajo básico

Crear repositorios, añadir cambios, hacer commits y push.



Colaborar con otros desarrolladores

Usar GitHub para ramas, pull requests y revisiones de código.

Control de Versiones

Introducción a Control de Versiones ¿Qué es?

El control de versiones es una práctica esencial en el desarrollo de software que permite rastrear y gestionar los cambios realizados en el código fuente a lo largo del tiempo.

Función:

Es registrar de forma precisa quién realizó cada modificación, qué cambios se introdujeron y cuándo se efectuaron. Esta capacidad de seguimiento detallado es crucial para la colaboración en equipo, ya que permite identificar y resolver conflictos de manera eficiente.



Tipos de Control de Versiones

Existen dos tipos principales de sistemas de control de versiones:

1 Centralizados:

En estos sistemas, como **SVN (Subversion)** y **CVS (Concurrent Versions System)**, todo el historial de versiones del código se almacena en un único servidor central. Los desarrolladores trabajan con una copia local del código, pero deben conectarse al servidor central para obtener las últimas actualizaciones y subir sus cambios. Esto simplifica la gestión para equipos pequeños, pero puede ser un punto de fallo único y dificulta el trabajo offline.

2 Descentralizados:

A diferencia de los sistemas centralizados, los sistemas descentralizados, como **Git**, permiten que cada desarrollador tenga una copia completa del repositorio en su máquina local. Esto facilita la colaboración, permite el trabajo offline, y hace que el sistema sea más robusto y menos susceptible a fallos del servidor. Git es el sistema más popular en la actualidad, y plataformas como GitHub y GitLab ofrecen servicios de alojamiento para repositorios Git y herramientas de colaboración.

Beneficios del Control de Versiones

Seguridad

Recupera versiones anteriores del código fácilmente si un archivo se corrompe, se elimina accidentalmente o se sobrescribe con cambios erróneos. Esto minimiza la pérdida de trabajo y permite la recuperación rápida.

Colaboración

Facilita la visibilidad de los cambios realizados por cada miembro del equipo. Permite una discusión clara sobre cada modificación, identificando posibles conflictos y mejorando la comunicación entre desarrolladores.

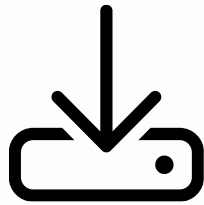
Confianza

Proporciona un registro preciso y auditable de todos los cambios en el código a lo largo del tiempo. Esto aumenta la confianza en la integridad del proyecto y facilita la resolución de problemas.

- ① Por ejemplo, imagina que dos desarrolladores trabajan simultáneamente en el mismo archivo. Sin control de versiones, los cambios de uno podrían sobrescribir los del otro, perdiendo parte del trabajo. Con un sistema de control de versiones, ambos pueden fusionar sus cambios sin conflictos, manteniendo un historial completo de todas las modificaciones.

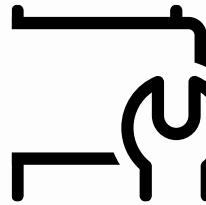
Instalación de Git

Git es un sistema de control de versiones distribuido, diseñado para manejar proyectos de cualquier tamaño con velocidad y eficiencia. Es gratuito, de código abierto y se ha convertido en el estándar para el desarrollo de software moderno. Antes de comenzar a trabajar con Git, necesitamos instalarlo siguiendo estos sencillos pasos:



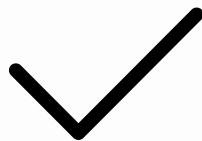
Descargar

Visitar git-scm.com/downloads y elegir el instalador para tu sistema.



Instalar

Seguir los pasos del instalador con las opciones por defecto.



Verificar

Abrir terminal y ejecutar 'git --
version' para confirmar la
instalación.





Configuración Inicial de Git

Antes de empezar a usar Git, es fundamental configurarlo correctamente. Esto asegura que tus commits estén correctamente asociados a tu identidad, lo cual es crucial para el seguimiento de cambios y la colaboración en proyectos.

```
git config --global user.name "Tu Nombre"  
git config --global user.email  
"tuemail@example.com"
```

Estos comandos asocian tus commits con tu identidad. Es importante para el trabajo en equipo.

Comandos Básicos Iniciales



git init

Crea un nuevo repositorio en una carpeta.



git status

Muestra el estado actual del repositorio.



git add

Prepara archivos modificados para el commit.



git commit

Registra cambios en el historial.

Un Ejemplo de Flujo de Trabajo con Git

Imagina que estás trabajando en un proyecto con otros desarrolladores. El proyecto consiste en desarrollar una aplicación web. Comienzas creando un nuevo repositorio con 'git init', para que Git pueda rastrear los cambios en el código. Luego, usas 'git add' para agregar los archivos que acabas de crear. Finalmente, usas 'git commit' para guardar esos cambios en el historial del proyecto.

Ahora, tu compañero de equipo hace cambios en el código. Para compartir tus cambios locales con el repositorio remoto (como GitHub o GitLab), usas el comando 'git push'. Esto sube tu commit al repositorio remoto, permitiendo que otros miembros del equipo accedan a las últimas modificaciones. Luego, tu compañero de equipo usa 'git pull' para descargar los cambios que subiste, y ¡listo! Ahora tienes los cambios más recientes en tu propio repositorio local.

Flujo de Trabajo Básico con Git

1 Inicializar

Crear repositorio con ``git init``

2 Preparar

Añadir cambios con ``git add``

3 Guardar

Confirmar con ``git commit -m "mensaje"``

4 Subir

Enviar al remoto con ``git push``

5 Actualizar

Descargar cambios con ``git pull``

Comandos Basicos

Git add

Estos comandos son fundamentales en el flujo de trabajo de Git, permitiendo registrar y compartir cambios. Sin embargo, el comando `git add` tiene una variante muy útil: `git add .`

El comando `git add .` añade **todos** los archivos modificados en el directorio actual (y sus subdirectorios) al staging area.

```
git add archivo.txt  
git commit -m "Descripción breve del  
cambio"  
git push origin main
```

Git add

Imagina que has modificado los archivos `index.html`, `style.css` y `script.js`. En lugar de usar `git add` para cada archivo individualmente:

```
git add index.html  
git add style.css  
git add script.js
```

Puedes usar `git add .` para añadirlos todos al staging area de una sola vez. Recuerda revisar que solo los archivos deseados estén en el staging area antes de realizar el `commit`.

Git commit



El comando `git commit` guarda los cambios del *staging area* en el historial local de tu repositorio Git. Es crucial para registrar el progreso y preservar las modificaciones.

Sintaxis:

```
git commit -m "Tu mensaje descriptivo"
```

Ejemplo:

```
git commit -m "Arreglé un error en la función principal"
```

Recuerda que un mensaje de commit claro y preciso facilita la comprensión del cambio realizado. Un buen mensaje describe brevemente qué se modificó y por qué.

Git push

`git push` envía los cambios del repositorio local al repositorio remoto, como GitHub. Permite a otros colaboradores obtener los últimos cambios y trabajar en conjunto.

`git push origin main` (o la rama correspondiente). Asegúrate de tener configurado tu repositorio remoto.

Sintaxis:

```
git push origin <nombre_de_la_rama>
```

Reemplaza `<nombre_de_la_rama>` con el nombre de la rama que deseas subir (ej: `main`, `develop`). Si no se especifica la rama, Git intentará empujar la rama actual.

Ejemplo:

```
git push origin main
```

Envía los cambios de la rama ``main`` al repositorio remoto.

```
git push origin feature/nueva-  
funcionalidad
```

Envía los cambios de la rama ``feature/nueva-funcionalidad`` al repositorio remoto.

Comandos Básicos: git pull

`git pull` actualiza tu repositorio local con los cambios del repositorio remoto.

`git pull origin main` (o la rama correspondiente) descarga las últimas actualizaciones de la rama `main` (o la rama que desees) desde el repositorio remoto `origin` y las integra en tu repositorio local.

Sintaxis:

```
git pull origin
```

Ejemplo:

```
git pull origin main
```

Actualiza la rama `main` local con los cambios de la rama `main` remota

```
git pull origin feature/nueva-  
funcionalidad
```

Actualiza la rama `feature/nueva-funcionalidad` local con los cambios de la rama remota del mismo nombre

Ramas

Ramas (Branches)

Las ramas en Git son como caminos paralelos en un proyecto. Permiten trabajar en nuevas características o correcciones de errores sin afectar el código principal. Cada rama es una copia independiente del repositorio, lo que permite experimentar y realizar cambios sin riesgo de dañar el código estable..

Crear una rama:

```
git branch nueva-funcionalidad
```

Cambiar a una rama:

```
git checkout nueva-funcionalidad
```



Github

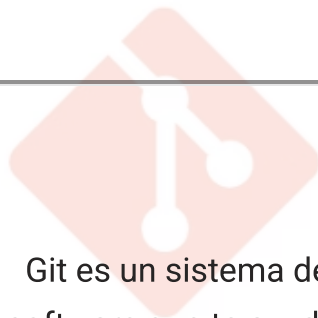
Introducción a GitHub

GitHub es una plataforma de desarrollo de software que permite a los usuarios trabajar en proyectos colaborativos de código abierto o privado.

GitHub funciona como un repositorio centralizado donde los desarrolladores pueden guardar, versionar y compartir su código fuente. Esta plataforma facilita la colaboración entre equipos, ya que permite a los miembros trabajar en el mismo proyecto simultáneamente sin sobrescribirse.

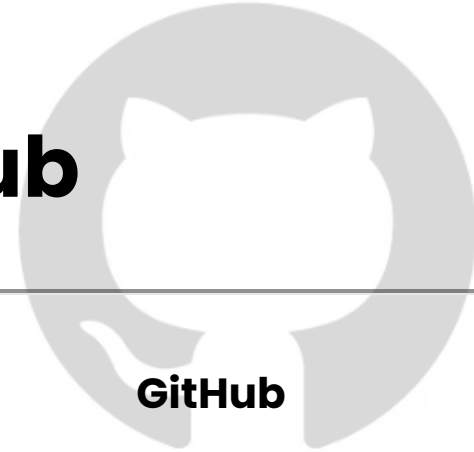


Diferencia entre Git y GitHub



Git

Git es un sistema de control de versiones. Es un software que te ayuda a rastrear los cambios en el código fuente.



GitHub

GitHub es una plataforma de desarrollo de software que utiliza Git para almacenar y administrar el código fuente.

Colaboración en Proyectos mediante GitHub



Fork

Crea una copia personal del repositorio original, llamada "fork". Esto te permite trabajar en tus propios cambios sin afectar el repositorio original. Para realizar un fork, usa el botón "Fork" en la página del repositorio en GitHub.



Pull Request

Cuando terminas de trabajar en tus cambios, crea un "pull request" para solicitar que se fusionen tus cambios en el repositorio original. Después de hacer commit de tus cambios localmente, puedes crear un pull request en GitHub usando la interfaz web o con el comando `git push origin``.

Buenas Prácticas de Versionado



Mensajes de Commit Claros

Describir brevemente el cambio. Ejemplo: "Agrega cálculo de descuento para bebidas".



Commits Pequeños y Frecuentes

Facilitan revertir cambios y revisar el historial.



Uso de Ramas

Mantener la rama principal estable. Desarrollar nuevas funcionalidades en ramas separadas.

Materialles y Recursos Adicionales

1 Documentación Oficial de Git

Visita <https://git-scm.com/doc> para información detallada.

2 Guía de Git de Atlassian

[Tutoriales y explicaciones paso a paso.](#)

3 Videos en YouTube

Busca tutoriales visuales sobre Git y GitHub. <https://youtu.be/kEPF-MWGq1w?si=JHLbH1U-8w264PWn>



Preguntas para Reflexionar



Productividad en TechLab

¿Cómo mejora la productividad y colaboración el uso de Git y GitHub?



Branching Models

¿Cómo ayudan los branching models a mantener un flujo de trabajo ordenado?



Ventajas del Historial

¿Qué ventajas ofrece mantener un historial detallado de cambios del código?

Ejercicios Prácticos
